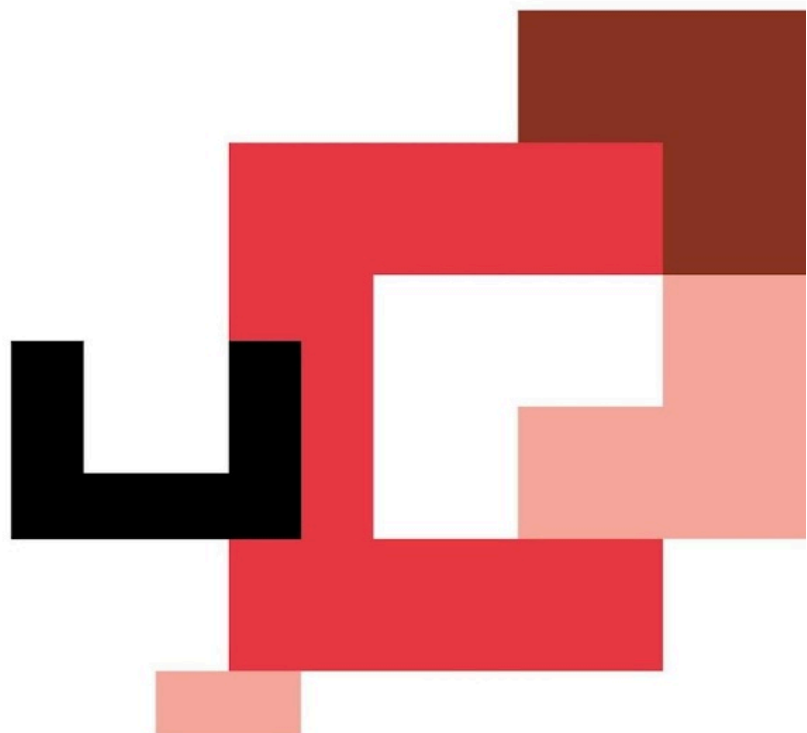




FULL STACK DEVELOPER + IA

Módulo 2 | Feature 4

Autenticación + Autorización
+ Seguridad



Objetivo de la feature

En esta feature vamos a añadir uno de los componentes más importantes de cualquier backend real: **la autenticación y autorización de usuarios**.

Hasta ahora nuestra API permitía acceder libremente a todos los endpoints. A partir de este sprint vamos a implementar un sistema que permita:

- Registrar usuarios
- Iniciar sesión
- Generar tokens de autenticación
- Proteger rutas
- Controlar permisos mediante roles

Además, reforzaremos la seguridad del servidor añadiendo **cabeceras seguras, control de dominios y limitación de peticiones**.

Aprenderás:

- Cómo funciona la **autenticación con JWT**
- Cómo proteger contraseñas usando **bcrypt**
- Cómo implementar **roles y permisos**
- Cómo proteger rutas con **middlewares**
- Cómo mejorar la seguridad de la API con **Helmet, CORS y Rate Limit**

El objetivo final es entender **cómo funciona el sistema de login de una API moderna**.

Requisitos

Tech / configuración

- Node.js **18+**
- Express
- Prisma ORM
- Supabase (PostgreSQL)

Nuevas dependencias:

- `jsonwebtoken (JWT)`
- `bcrypt`
- `cors`
- `helmet`
- `express-rate-limit`

Instalación:

Shell

```
npm install jsonwebtoken bcrypt cors helmet express-rate-limit
```

Variables de Entorno

En este sprint añadimos variables de seguridad.

Archivo `.env`:

None

```
DATABASE_URL="postgresql://..."

JWT_SECRET="tu_clave_secreta_super_segura"

JWT_EXPIRES_IN="7d"

PORT=3000
```

Importante

Si `JWT_SECRET` **no está definido**, el servidor **no debe arrancar**.

Esto evita errores de seguridad.

Estructura del Proyecto

None

```
src/

├─ config/

├─ controllers/

|   └─ auth.controller.js
```

```
|   └─ users.controller.js
└─ services/
    |   └─ auth.service.js
    |   └─ users.service.js
└─ routes/
    |   └─ auth.routes.js
    |   └─ users.routes.js
    |   └─ products.routes.js
└─ middlewares/
    |   └─ authenticate.js
    |   └─ requireRole.js
    |   └─ validateProduct.js
    |   └─ errorHandler.js
    |   └─ notFound.js
└─ app.js
└─ server.js
```

```
prisma/
```

```
└─ schema.prisma
```

```
.env
```

Endpoints

Auth (Públicos)

Método	Ruta	Descripción
POST	/api/auth/register	Registrar usuario
POST	/api/auth/login	Login y generación de token

Productos

Método	Ruta	Acceso
GET	/api/products	Público
GET	/api/products/:id	Público
POST	/api/products	ADMIN
PUT	/api/products/:id	ADMIN
DELETE	/api/products/:id	ADMIN



Usuarios

Método	Ruta	Descripción
GET	<code>/api/users/profile</code>	Perfil del usuario autenticado

Qué hay que hacer (paso a paso)

1) Implementar registro de usuario

Endpoint:

```
None
POST /api/auth/register
```

Debe:

- Recibir `email` y `password`
- Aplicar `bcrypt.hash()`
- Guardar el usuario en la base de datos

Nunca guardar la contraseña en texto plano.

2) Implementar login

Endpoint:

```
None
POST /api/auth/login
```

Debe:

1. Buscar usuario por email
2. Comparar password con `bcrypt.compare()`
3. Generar token JWT

Ejemplo conceptual:

```
JavaScript  
jwt.sign(payload, JWT_SECRET, { expiresIn })
```

3) Crear middleware authenticate

Este middleware:

- Lee el header:

```
None  
Authorization: Bearer TOKEN
```

- - Verifica el token con `jwt.verify()`
- Añade el usuario al request:

```
JavaScript  
req.user
```

Si el token es inválido → **401 Unauthorized**

4) Crear middleware requireRole

Este middleware permite restringir rutas.

Ejemplo:

```
JavaScript  
requireRole("ADMIN")
```

Solo usuarios con ese rol podrán acceder.

Si no tiene permiso → **403 Forbidden**

5) Proteger rutas de productos

Las rutas de modificación deben estar protegidas.

Ejemplo:

```
None
POST /api/products

PUT /api/products/:id

DELETE /api/products/:id
```

Solo usuarios **ADMIN**.

Seguridad Web

En esta feature añadimos tres herramientas clave.

Helmet

Añade cabeceras HTTP seguras automáticamente.

Ejemplo:

```
None
X-DNS-Prefetch-Control

X-Frame-Options

X-XSS-Protection
```

Uso:




```
JavaScript
app.use(helmet())
```

CORS

Controla qué dominios pueden acceder a la API.

Ejemplo:

```
JavaScript
app.use(cors())
```

En producción se debe restringir.

Rate Limit

Limita el número de peticiones por IP.

Ejemplo:

```
None
100 requests / 15 minutos
```

Evita ataques de fuerza bruta en login.

Cómo usar Authorization Bearer

- 1 - Hacer login
- 2 - Copiar el token recibido
- 3 - Enviar en la cabecera:

None

Authorization: Bearer <TOKEN>

Uso de la IA en este sprint

La IA puede ayudarte a entender cómo funcionan los sistemas de autenticación.

Puedes usarla para:

- Revisar el flujo de login
 - Verificar el uso correcto de JWT
 - Detectar problemas de seguridad
-

Prompts útiles

Explicación de JWT

None

Explícame cómo funciona JWT en una API Express
y qué datos debería incluir en el payload.

Revisión de seguridad

None

Estoy implementando login con JWT y bcrypt en Express.
¿Qué errores de seguridad debería evitar?

Debug de autenticación

None

Tengo este error verificando JWT en Express.

Te paso el middleware authenticate.

Dime las causas más probables.

Pistas

1. No guardar passwords en texto plano

Siempre usar:

None

```
bcrypt.hash()
```

2. El token no guarda la contraseña

El JWT debe contener solo información mínima.

Ejemplo:

None

```
userId
```

```
role
```

```
email
```

3. Los middlewares se ejecutan en orden

Ejemplo correcto:

None

`authenticate → requireRole → controller`

Importante que tienen que tener en cuenta

1. Seguridad primero

Nunca subir:

None

`.env`

2. Los tokens expiran

JWT debe tener expiración.

Ejemplo:

None

`7d`

3. Separación de capas

La arquitectura sigue siendo:

None

`routes → controllers → services → prisma`

4. Los roles controlan acceso

Un usuario normal **no debe poder modificar productos**.

Checks rápidos (autoevaluación)

El sprint está correcto si:

- `POST /api/auth/register` crea usuario
 - `POST /api/auth/login` devuelve token
 - El token funciona en rutas protegidas
 - Un usuario USER no puede crear productos
 - Un ADMIN sí puede crear productos
 - Sin token → **401**
 - Sin permisos → **403**
-

Ejemplos cURL

Registrar usuario

Shell

```
curl -X POST http://localhost:3000/api/auth/register \
-H "Content-Type: application/json" \
-d '{"email":"user@test.com","password":"password123"}'
```

Login

Shell

```
curl -X POST http://localhost:3000/api/auth/login \
-H "Content-Type: application/json" \
-d '{"email":"user@test.com","password":"password123"}'
```

Guarda el token:



None

```
TOKEN="eyJhbGciOi.."
```

Ver perfil

Shell

```
curl http://localhost:3000/api/users/profile \  
-H "Authorization: Bearer $TOKEN"
```

Crear producto (ADMIN)

Shell

```
curl -X POST http://localhost:3000/api/products \  
-H "Content-Type: application/json" \  
-H "Authorization: Bearer $TOKEN_ADMIN" \  
-d '{"name":"Cazadora Cuero","price":129.99,"stock":15}'
```

Intentar crear sin token

Shell

```
curl -X POST http://localhost:3000/api/products \  
-H "Content-Type: application/json" \  
-d '{"name":"Test","price":10}'
```

Respuesta esperada:



None

401 Unauthorized

Intentar crear con rol USER

Shell

```
curl -X POST http://localhost:3000/api/products \  
-H "Content-Type: application/json" \  
-H "Authorization: Bearer $TOKEN_USER" \  
-d '{"name":"Test","price":10}'
```

Respuesta esperada:

None

403 Forbidden

Notas

- `passwordHash` ahora es obligatorio en el modelo `User`.
- `authenticate` añade `req.user`.
- `requireRole()` puede aceptar múltiples roles.
- Si `JWT_SECRET` no existe el servidor **no debe arrancar**.